



ADITYA UNIVERSITY

Department of Computer Science and Engineering

DBMS CORNERSTONE PROJECT

PROJECT DOCUMENTATION

HOTEL RESERVATION MANAGEMENT SYSTEM

Submitted by

Student Name	Roll Number
G. Keerthi Harshana	25B11CS269
T. Radhika	25B11CS945
T. Lakshmi Sowjanya	26B21CS070
R. Pavithra	25B11CS804

Under the guidance of

Mrs. Kasichainula Vydehi, M.Tech.,(Ph.D),
Assistant Professor,
Department of CSE

CERTIFICATE

This is to certify that the project documentation titled "Hotel Reservation Management System" is a bonafide record of the DBMS Cornerstone Project work carried out by the following students of the Department of Computer Science and Engineering, Aditya University.

Student Name	Roll Number
G. Keerthi Harshana	25B11CS269
T. Radhika	25B11CS945
T. Lakshmi Sowjanya	26B21CS070
R. Pavithra	25B11CS804

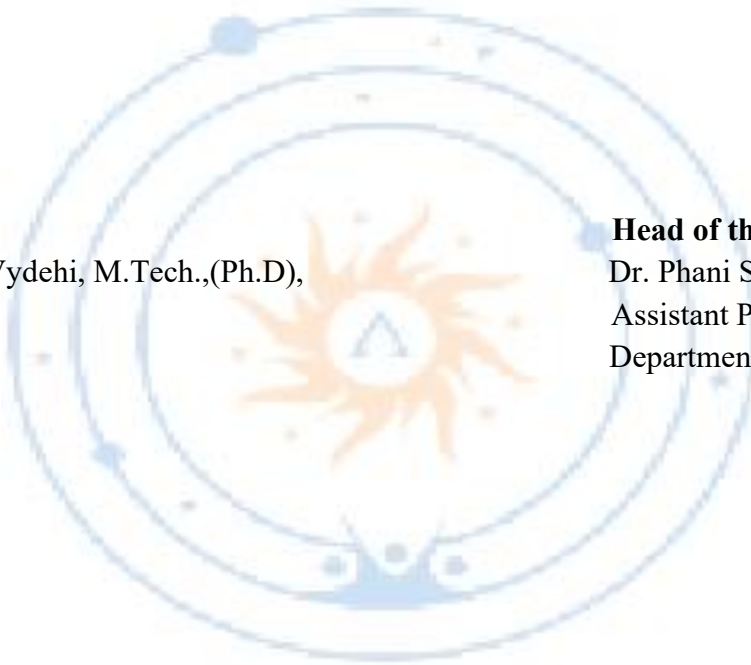
The work embodied in this documentation covers the design, normalization and SQL implementation of a relational database for hotel reservation management, and has been carried out under my supervision.

Project Guide

Mrs. Kasichainula Vydehi, M.Tech.,(Ph.D),
Assistant Professor,
Department of CSE

Head of the department

Dr. Phani Sridhar Addepalli,Ph.D,
Assistant Professor,
Department of CSE



ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, [Guide Name], for the valuable guidance, encouragement and constructive feedback provided throughout the course of this DBMS Cornerstone Project.

We are thankful to the Head of the Department and the faculty members of the Department of Computer Science and Engineering, Aditya University, for providing the facilities and academic support needed to complete this work.

We also thank our friends and families for their continuous support and motivation. This project helped us apply the concepts of database design, normalization and SQL to a practical hotel reservation scenario.

Project Team

G. Keerthi Harshana (25B11CS269)
T. Radhika (25B11CS945)
T. Lakshmi Sowjanya (26B21CS070)
R. Pavithra (25B11CS804)

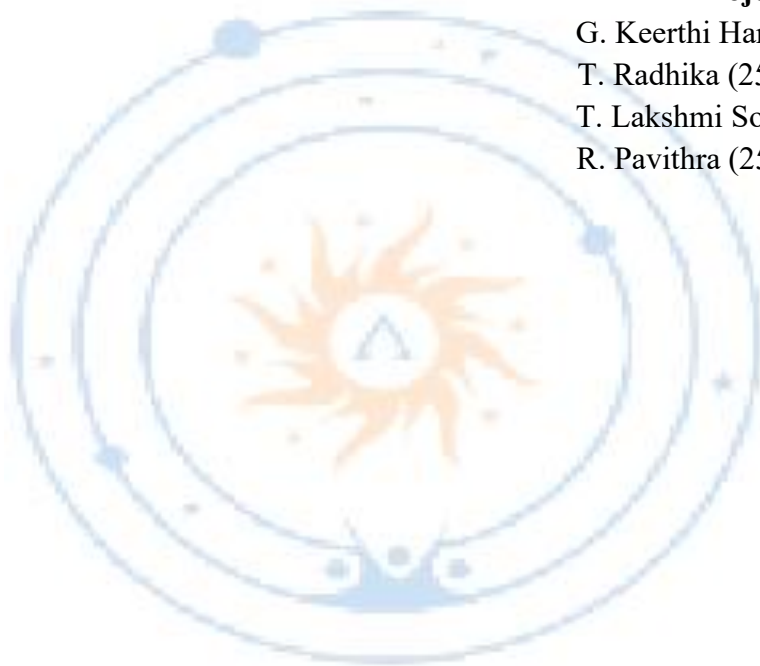
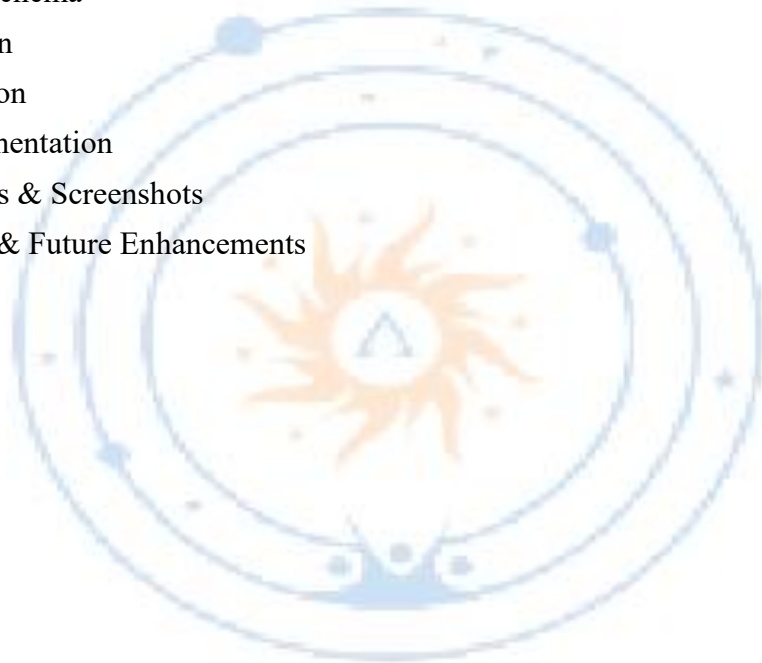


TABLE OF CONTENTS

1. Cover Page
2. Certificate
3. Acknowledgement
4. Table of Contents
5. Abstract
6. Introduction
7. System Analysis
8. Database Design
9. ER Diagram
10. Relational Schema
11. Table Design
12. Normalization
13. SQL Implementation
14. SQL Queries & Screenshots
15. Conclusion & Future Enhancements
16. References



ABSTRACT

The Hotel Reservation Management System is a database application designed to manage hotel bookings through a centralized relational database. The system stores and manages information related to hotels, rooms, customers, reservations, payments and staff. It replaces time-consuming and error-prone manual booking records with a structured Oracle SQL database that supports quick room-availability checks, reservation management and organized record keeping.

The database consists of six related tables – HOTEL, CUSTOMER, ROOM, RESERVATION, PAYMENT and STAFF – connected through primary and foreign keys and protected by NOT NULL, UNIQUE, CHECK and DEFAULT constraints. The design is normalized up to Third Normal Form (3NF) to reduce redundancy and update anomalies, and is demonstrated with sample records and SQL queries covering selection, joins, aggregation and reporting.



LIST OF FIGURES

Figure 1. ER Diagram of Hotel Reservation Management System

Figure 2. Tables created in Oracle Database

Figure 3. Sample records – HOTEL table

Figure 4. Sample records – ROOM table

Figure 5. Sample records – STAFF table

Figure 6. Sample records – RESERVATION table

Figure 7. Sample records – CUSTOMER table

Figure 8. Output of Q1

Figure 9. Output of Q2

Figure 10. Output of Q3

Figure 11. Output of Q4

Figure 12. Output of Q5

Figure 13. Output of Q7



LIST OF TABLES

Table 1. Functional Requirements
Table 2. End Users of the System
Table 3. Software Requirements
Table 4. Major Database Entities
Table 5. Relationships between Entities
Table 6. Relational Schema
Table 7. Hotel Table Structure
Table 8. Customer Table Structure
Table 9. Room Table Structure
Table 10. Reservation Table Structure
Table 11. Payment Table Structure
Table 12. Staff Table Structure
Table 13. Keys and Constraints
Table 14. 1NF – CUSTOMER Table
Table 15. 1NF – RESERVATION Table
Table 16. 2NF – CUSTOMER Table
Table 17. 2NF – ROOM Table
Table 18. 2NF – RESERVATION Table
Table 19. 3NF – Functional Dependencies
Table 20. 3NF – STAFF Table
Table 21. 3NF – HOTEL Table
Table 22. Relations after Normalization

ACRONYMS

- DBMS – Database Management System
- ER – Entity Relationship
- SQL – Structured Query Language
- DDL – Data Definition Language
- DML – Data Manipulation Language
- DQL – Data Query Language
- PK – Primary Key
- FK – Foreign Key
- 1NF – First Normal Form
- 2NF – Second Normal Form
- 3NF – Third Normal Form



1. INTRODUCTION

1.1 Brief Information about the Project

A hotel reservation management system helps customers book rooms easily and efficiently. It allows users to check room availability and select suitable rooms, and it stores customer and reservation details securely. The system reduces manual work and minimizes booking errors, and provides an organized way to manage bookings, cancellations and customer information.

1.2 Motivation and Problem Statement

Manual hotel bookings are time-consuming and error-prone. Checking room availability manually is difficult, managing customer and booking details can be inefficient, and booking conflicts may occur due to poor record management. A computerized system is therefore needed to manage reservations efficiently.

The Hotel Reservation Management System is developed to overcome these problems by storing and managing hotel-related information in a centralized database. The system helps manage customer details, hotel information, room details, reservations, payments and staff information efficiently.

1.3 Objectives of the Project

The main objectives of the Hotel Reservation Management System are:

- To manage customer information efficiently.
- To manage hotel and room details.
- To check room availability quickly.
- To create, update and cancel room reservations.
- To maintain payment information.
- To reduce manual work and paperwork.
- To minimize errors and duplicate bookings.
- To maintain accurate and consistent records.
- To provide quick access to customer, room and reservation information.
- To improve the overall efficiency of hotel management.

1.4 Scope of the Project

The scope covers database-oriented management of hotel reservations. The core scope includes customer, hotel, room, reservation, payment and staff data, together with the SQL queries needed to retrieve and summarize this information. The database can serve as the backend for a reservation application used by guests, receptionists, hotel managers and administrators.

2. SYSTEM ANALYSIS

2.1 Existing System

- Hotel bookings are handled manually, which is time-consuming and error-prone.
- Checking room availability by hand is difficult.
- Customer and booking details are managed inefficiently.
- Booking conflicts may occur due to poor record management.
- Cancellations and changes are hard to track.

2.2 Proposed System

The proposed system stores and manages hotel-related information in a centralized Oracle SQL database. It organizes data into related tables and uses primary keys, foreign keys, uniqueness rules, NOT NULL conditions, DEFAULT values and CHECK constraints to keep records accurate and consistent.

- Centralized database for hotels, rooms, customers, reservations, payments and staff
- Quick room-availability checking
- Reservation creation, modification and cancellation
- Payment record maintenance
- SQL-based reporting
- Referential integrity between related tables

2.3 Functional Requirements

Table 1: Functional Requirements

Module	Functional Requirement
User Management	User registration and login.
Room Search	Search available rooms by date and type. View room details, price and availability.
Reservation Management	Make and confirm room reservations. Modify or cancel reservations.
Customer Management	Store and manage customer information.
Billing and Payment	Calculate booking cost and generate bills. Process payment details securely.
Notifications	Send booking confirmation and notifications.
Administration	Admin can manage rooms, bookings and customers.

2.4 Non-Functional Requirements

- Usability – the system should be easy to use.
- Performance – the system should provide fast response time.
- Security – the system should ensure data security and provide secure user authentication.
- Reliability – the system should be reliable and available, with minimal downtime.
- Scalability – the system should be scalable for future needs and support multiple users.
- Accuracy – the system should maintain data accuracy.
- Maintainability – the system should be easy to maintain.

2.5 End Users of the System

Table 2: End Users of the System

User	Role in the System
Customer / Guest	Searches for rooms, checks availability, makes reservations and cancels bookings.
Receptionist	Manages customer check-in, check-out and reservations.
Hotel Manager	Monitors rooms, bookings, customers and hotel operations.
Administrator	Manages users, rooms, hotel details and system records.

2.6 Requirements Specification

Table 3: Software Requirements

Category	Requirement
Database	Oracle Database
Language	SQL (DDL, DML and DQL statements)



3. DATABASE DESIGN

3.1 Major Entities and Attributes

The database is built around six entities. Each entity and its attributes are listed below.

Table 4: Major Database Entities

Entity	Attributes
1. CUSTOMER	Customer_ID, Name, Phone, Email, Address
2. ROOM	Room_ID, Room_Number, Room_Type, Price, Status
3. RESERVATION	Reservation_ID, Customer_ID, Room_ID, Check_In_Date, Check_Out_Date, Booking_Status
4. STAFF	Staff_ID, Staff_Name, Phone, Role, Salary
5. HOTEL	Hotel_ID, Hotel_Name, Location, Phone, Email
6. PAYMENT	Payment_ID, Reservation_ID, Payment_Date, Amount, Payment_Method, Payment_Status

3.2 ER Model

The ER model represents the six entities and the relationships between them. A hotel has many rooms and employs many staff members; a customer can make many reservations; a room can be booked in many reservations; and a reservation can be associated with payment records. The ER diagram in the next section shows the entities, their attributes, and the primary and foreign keys.

3.3 Relationships

Table 5: Relationships between Entities

Relationship	Description
HOTEL → ROOM	One hotel has many rooms (ROOM.Hotel_ID references HOTEL).
HOTEL → STAFF	One hotel employs many staff members (STAFF.Hotel_ID references HOTEL).
CUSTOMER → RESERVATION	One customer can make many reservations (RESERVATION.Customer_ID references CUSTOMER).
ROOM → RESERVATION	One room can appear in many reservations over time (RESERVATION.Room_ID references ROOM).
RESERVATION → PAYMENT	One reservation can have payment records (PAYMENT.Reservation_ID references RESERVATION).

4. ER DIAGRAM

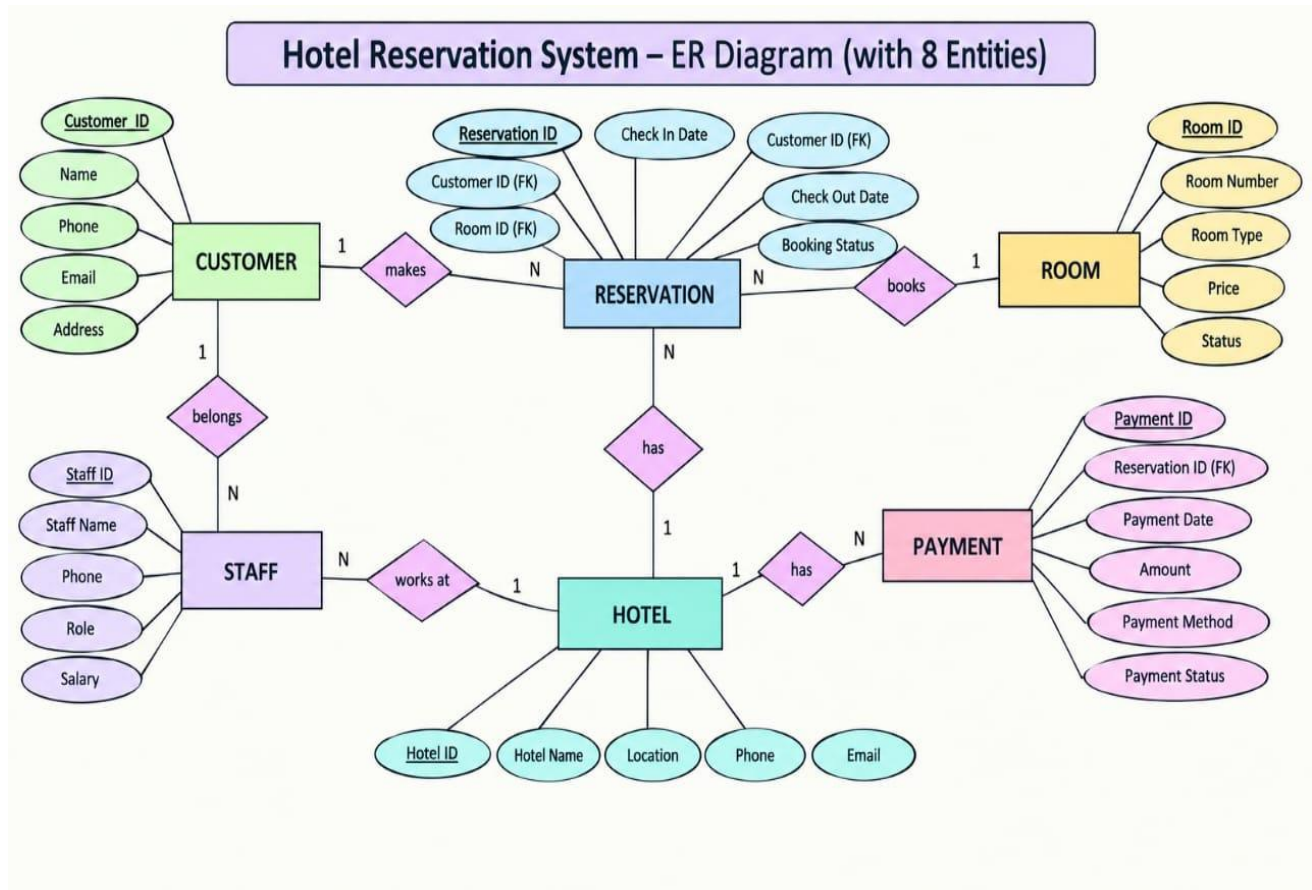


Figure 1: ER Diagram of Hotel Reservation Management System

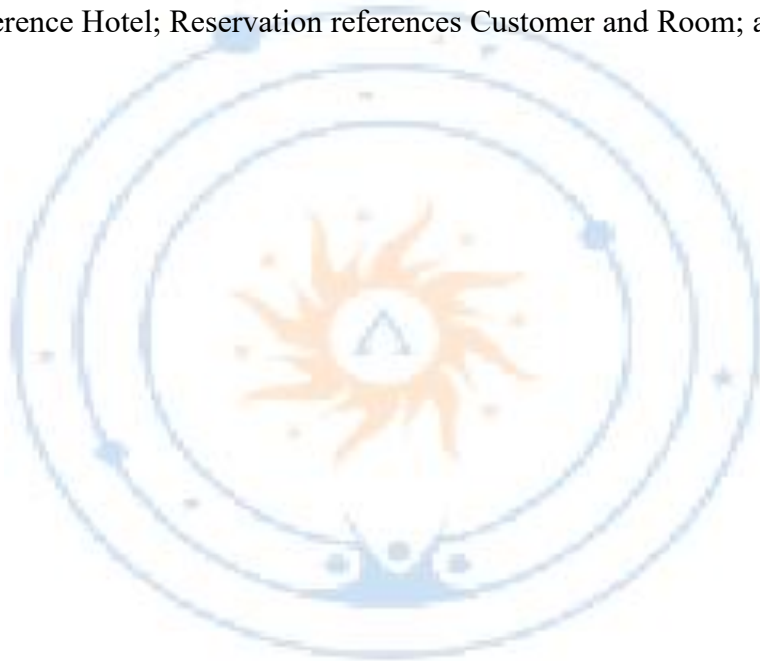
The ER diagram shows the six entities of the system – HOTEL, ROOM, STAFF, CUSTOMER, RESERVATION and PAYMENT – with their attributes. Primary keys are shown in bold and foreign keys are marked (FK). The labels 1 and N indicate one-to-many cardinality.

5. RELATIONAL SCHEMA

Table 6: Relational Schema

Relation	Relational Schema
HOTEL	Hotel_ID (PK), Hotel_Name, Location, Phone, Email
CUSTOMER	Customer_ID (PK), Name, Phone, Email, Address
ROOM	Room_ID (PK), Room_Number, Room_Type, Price, Status, Hotel_ID (FK)
RESERVATION	Reservation_ID (PK), Customer_ID (FK), Room_ID (FK), Check_In_Date, Check_Out_Date, Booking_Status
PAYMENT	Payment_ID (PK), Reservation_ID (FK), Payment_Date, Amount, Payment_Method, Payment_Status
STAFF	Staff_ID (PK), Staff_Name, Phone, Role, Salary, Hotel_ID (FK)

Room and Staff reference Hotel; Reservation references Customer and Room; and Payment references Reservation.



6. TABLE DESIGN

6.1 Hotel Table

Table 7: Hotel Table Structure

Attribute	Data Type	Constraint	Description
Hotel_ID	NUMBER	PRIMARY KEY	Unique hotel identifier
Hotel_Name	VARCHAR2(50)	NOT NULL	Name of the hotel
Location	VARCHAR2(50)	NOT NULL	City / location of the hotel
Phone	VARCHAR2(15)	UNIQUE	Hotel contact number
Email	VARCHAR2(60)	UNIQUE	Hotel email address

6.2 Customer Table

Table 8: Customer Table Structure

Attribute	Data Type	Constraint	Description
Customer_ID	NUMBER	PRIMARY KEY	Unique customer identifier
Name	VARCHAR2(50)	NOT NULL	Customer name
Phone	VARCHAR2(15)	UNIQUE	Customer contact number
Email	VARCHAR2(60)	UNIQUE	Customer email address
Address	VARCHAR2(100)	—	Customer address

6.3 Room Table

Table 9: Room Table Structure

Attribute	Data Type	Constraint	Description
Room_ID	NUMBER	PRIMARY KEY	Unique room identifier
Room_Number	VARCHAR2(10)	UNIQUE, NOT NULL	Room number
Room_Type	VARCHAR2(30)	NOT NULL	Single, Double, Deluxe or Suite
Price	NUMBER(10,2)	CHECK (Price > 0) DEFAULT 'Available';	Room price
Status	VARCHAR2(20)	CHECK (Available, Occupied, Maintenance)	Current room status
Hotel_ID	NUMBER	NOT NULL, FK	References HOTEL

6.4 Reservation Table

Table 10: Reservation Table Structure

Attribute	Data Type	Constraint	Description
Reservation_ID	NUMBER	PRIMARY KEY	Unique reservation identifier

Attribute	Data Type	Constraint	Description
Customer_ID	NUMBER	NOT NULL, FK	References CUSTOMER
Room_ID	NUMBER	NOT NULL, FK	References ROOM
Check_In_Date	DATE	NOT NULL	Check-in date
Check_Out_Date	DATE	NOT NULL; CHECK (after Check_In_Date)	Check-out date
Booking_Status	VARCHAR2(20)	DEFAULT 'Confirmed'	Confirmed / Cancelled etc.

6.5 Payment Table

Table 11: Payment Table Structure

Attribute	Data Type	Constraint	Description
Payment_ID	NUMBER	PRIMARY KEY	Unique payment identifier
Reservation_ID	NUMBER	NOT NULL, FK	References RESERVATION
Payment_Date	DATE	—	Date of payment
Amount	NUMBER(10,2)	CHECK (Amount > 0)	Amount paid
Payment_Method	VARCHAR2(30)	—	Mode of payment
Payment_Status	VARCHAR2(20)	—	Paid / Pending etc.

6.6 Staff Table

Table 12: Staff Table Structure

Attribute	Data Type	Constraint	Description
Staff_ID	NUMBER	PRIMARY KEY	Unique staff identifier
Staff_Name	VARCHAR2(50)	NOT NULL	Staff name
Phone	VARCHAR2(15)	UNIQUE	Staff contact number
Role	VARCHAR2(30)	NOT NULL	Manager, Receptionist, etc.
Salary	NUMBER(10,2)	CHECK (Salary > 0)	Staff salary
Hotel_ID	NUMBER	NOT NULL, FK	References HOTEL

6.7 Keys and Constraints

Table 13: Keys and Constraints

Constraint Type	Applied To / Purpose
Primary Key	Hotel_ID, Customer_ID, Room_ID, Reservation_ID, Payment_ID, Staff_ID
Foreign Key	Room.Hotel_ID; Staff.Hotel_ID; Reservation.Customer_ID; Reservation.Room_ID; Payment.Reservation_ID
NOT NULL	Required attributes such as names, room number, room type, dates and foreign keys
UNIQUE	Phone and email values, and Room_Number

Constraint Type	Applied To / Purpose
CHECK	Price > 0; Salary > 0; Room Status in (Available, Occupied, Maintenance); Check_Out_Date > Check_In_Date
DEFAULT	Room Status = 'Available'; Booking_Status = 'Confirmed'



7. Normalisation

1NF – First Normal Form

Contains atomic (single) values

Avoids repeating groups

Each record has a unique primary key

2NF – Second Normal Form

Must already satisfy 1NF

Eliminates partial dependency

Every attribute depends on the entire primary key

3NF – Third Normal Form

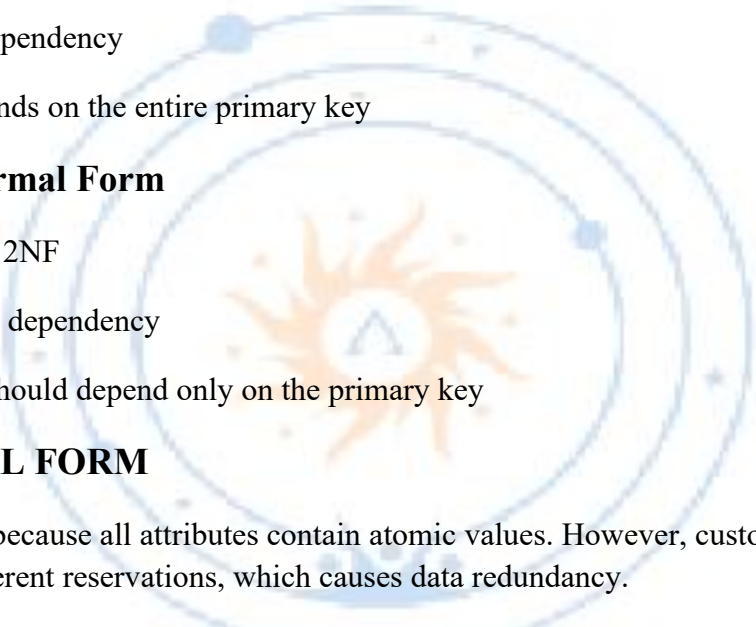
Must already satisfy 2NF

Eliminates transitive dependency

Non-key attributes should depend only on the primary key

FIRST NORMAL FORM

The table is in 1NF because all attributes contain atomic values. However, customer and room details are repeated for different reservations, which causes data redundancy.



RESERVATION_ID	CUSTOMER_ID	CUSTOMER_NAME	ROOM_ID	ROOM_TYPE
R1	C1	Ravi	RM1	Deluxe
R2	C2	Sita	RM2	Standard
R3	C3	Arjun	RM3	Suite
R4	C1	Ravi	RM2	Standard
R5	C4	Priya	RM4	Deluxe
R6	C2	Sita	RM3	Suite

Primary Key: (RESERVATION_ID, ROOM_ID)

RESERVATION_ID → CUSTOMER_ID, ROOM_ID
CUSTOMER_ID → CUSTOMER_NAME
ROOM_ID → ROOM_TYPE

- **Redundancy in 1NF**
- Customer details are repeated for C1 and C2.
- Room type is repeated whenever the same room occurs in another reservation.
- Repeated data increases storage and inconsistency.
- **Anomalies in 1NF**
- **Update Anomaly:**
If Ravi's name/details change, all rows containing C1 must be updated.
- **Insert Anomaly:**
A new customer cannot be stored until a reservation is created.
- **Delete Anomaly:**
Deleting the last reservation of a customer may also remove that customer's information.
- **Second Normal Form**
- 2NF – Decomposition
- The 1NF table has a composite primary key. Some attributes depend only on part of the composite key. Therefore, partial dependencies are removed by decomposing the table.
- **CUSTOMER details separated**

CUSTOMER_ID	CUSTOMER_NAME
C1	Ravi
C2	Sita
C3	Arjun
C4	Priya

Primary Key: CUSTOMER_ID

CUSTOMER_ID → CUSTOMER_NAME ROOM_ID → ROOM_TYPE

Reservation table after removing CUSTOMER_NAME

RESERVATION_ID	CUSTOMER_ID	ROOM_ID	ROOM_TYPE
R1	C1	RM1	Deluxe
R2	C2	RM2	Standard
R3	C3	RM3	Suite
R4	C1	RM2	Standard
R5	C4	RM4	Deluxe
R6	C2	RM3	Suite

CUSTOMER_NAME is deleted from the reservation table because it depends on CUSTOMER_ID, not on the complete composite key.

Redundancy after 2NF

- Customer names are stored only once.
- Repetition of customer information is reduced.
- Room information is still repeated where the same room occurs.

Anomalies after 2NF

- **Update anomaly:** Customer name needs to be updated only once.
- **Insert anomaly:** A customer can be added independently.
- **Delete anomaly:** Deleting a reservation does not delete customer information.

- **Third Normal Form**
- 2NF → 3NF Decomposition
- The 2NF table still contains a transitive dependency because $\text{ROOM_ID} \rightarrow \text{ROOM_TYPE}$. Therefore, room details are separated into a new ROOM table.
- **ROOM**

ROOM_ID	ROOM_TYPE
RM1	Deluxe
RM2	Standard
RM3	Suite
RM4	Deluxe

Primary Key: ROOM_ID

$\text{CUSTOMER_ID} \rightarrow \text{CUSTOMER_NAME}$ $\text{ROOM_ID} \rightarrow \text{ROOM_TYPE}$ ($\text{RESERVATION_ID}, \text{ROOM_ID}$) $\rightarrow \text{CUSTOMER_ID}$

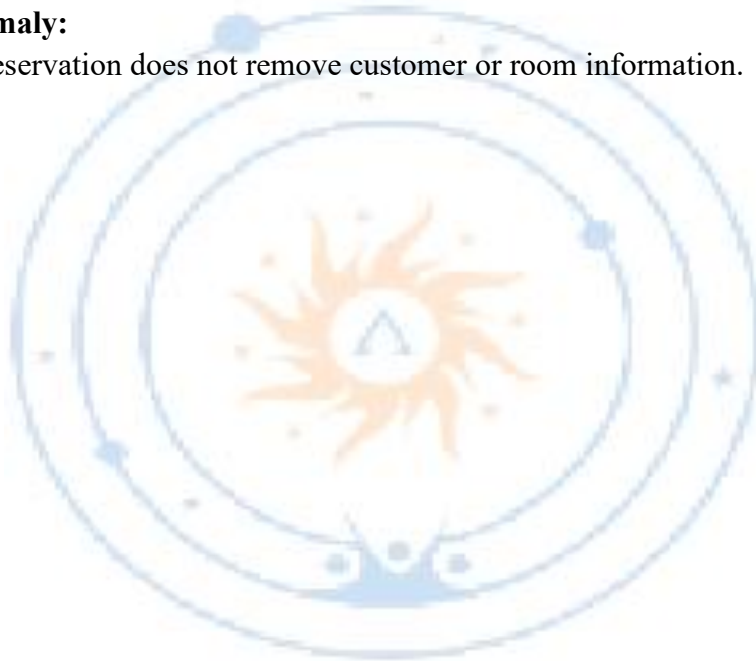
Final RESERVATION table

RESERVATION_ID	CUSTOMER_ID	ROOM_ID
R1	C1	RM1
R2	C2	RM2
R3	C3	RM3
R4	C1	RM2
R5	C4	RM4
R6	C2	RM3

Primary Key: RESERVATION_ID, ROOM_ID

- **Redundancy after 3NF**

- Customer information is stored only in CUSTOMER.
- Room information is stored only in ROOM.
- Repeated customer and room details are eliminated.
- Data duplication is minimized.
- **Anomalies after 3NF**
- **Update Anomaly:**
Customer or room details are updated in only one place.
- **Insert Anomaly:**
New customers and rooms can be added without creating a reservation.
- **Delete Anomaly:**
Deleting a reservation does not remove customer or room information.



8. SQL IMPLEMENTATION

8.1 DDL – Data Definition Language

The tables are created in Oracle Database using the following DDL statements. The parent tables HOTEL and CUSTOMER are created first, followed by the tables that reference them.

```
CREATE TABLE HOTEL (  
    Hotel_ID NUMBER PRIMARY KEY,  
    Hotel_Name VARCHAR2(50) NOT NULL,  
    Location VARCHAR2(50) NOT NULL,  
    Phone VARCHAR2(15) UNIQUE,  
    Email VARCHAR2(60) UNIQUE  
);
```

```
CREATE TABLE CUSTOMER (  
    Customer_ID NUMBER PRIMARY KEY,  
    Name VARCHAR2(50) NOT NULL,  
    Phone VARCHAR2(15) UNIQUE,  
    Email VARCHAR2(60) UNIQUE,  
    Address VARCHAR2(100)  
);
```

```
CREATE TABLE ROOM (  
    Room_ID NUMBER PRIMARY KEY,  
    Room_Number VARCHAR2(10) UNIQUE NOT NULL,  
    Room_Type VARCHAR2(30) NOT NULL,  
    Price NUMBER(10,2) CHECK (Price > 0),  
    Status VARCHAR2(20) DEFAULT 'Available'  
        CHECK (Status IN ('Available', 'Occupied', 'Maintenance')),  
    Hotel_ID NUMBER NOT NULL,  
    FOREIGN KEY (Hotel_ID) REFERENCES HOTEL(Hotel_ID)  
);
```

```
CREATE TABLE RESERVATION (  
    Reservation_ID NUMBER PRIMARY KEY,  
    Customer_ID NUMBER NOT NULL,  
    Room_ID NUMBER NOT NULL,  
    Check_In_Date DATE NOT NULL,  
    Check_Out_Date DATE NOT NULL,  
    Booking_Status VARCHAR2(20) DEFAULT 'Confirmed',  
    FOREIGN KEY (Customer_ID) REFERENCES CUSTOMER(Customer_ID),  
    FOREIGN KEY (Room_ID) REFERENCES ROOM(Room_ID),  
    CHECK (Check_Out_Date > Check_In_Date)  
);
```

```
CREATE TABLE PAYMENT (  

```

```

Payment_ID NUMBER PRIMARY KEY,
Reservation_ID NUMBER NOT NULL,
Payment_Date DATE,
Amount NUMBER(10,2) CHECK (Amount > 0),
Payment_Method VARCHAR2(30),
Payment_Status VARCHAR2(20),
FOREIGN KEY (Reservation_ID) REFERENCES RESERVATION(Reservation_ID)
);

```

```

CREATE TABLE STAFF (
Staff_ID NUMBER PRIMARY KEY,
Staff_Name VARCHAR2(50) NOT NULL,
Phone VARCHAR2(15) UNIQUE,
Role VARCHAR2(30) NOT NULL,
Salary NUMBER(10,2) CHECK (Salary > 0),
Hotel_ID NUMBER NOT NULL,
FOREIGN KEY (Hotel_ID) REFERENCES HOTEL(Hotel_ID)
);

```

After execution, six relational tables were successfully created in Oracle Database.



TABLE_NAME
CUSTOMER
HOTEL
PAYMENT
RESERVATION
ROOM
STAFF

6 relational tables were successfully created in Oracle Database.

Figure 2: Tables created in Oracle Database

8.2 Sample Records

Sample records were inserted into the tables and displayed using SELECT statements.

Hotel Table

SELECT * FROM HOTEL;

HOTEL_ID	HOTEL_NAME	LOCATION	PHONE	EMAIL
1	Royal Inn	Hyderabad	9876500001	royal@gmail.com
2	Green Park	Vijayawada	9876500002	green@gmail.com
3	Grand Palace	Guntur	9876500003	grand@gmail.com
4	City Comfort	Chennai	9876500004	city@gmail.com
5	Blue Moon	Vizag	9876500005	blue@gmail.com
6	Sun Residency	Tirupati	9876500006	sun@gmail.com
7	Sai Krishna	Warangal	9876500007	sai@gmail.com
8	Lake View	Nellore	9876500008	lake@gmail.com
9	Silver Star	Kurnool	9876500009	silver@gmail.com
10	Paradise	Mumbai	9876500010	paradise@gmail.com
More than 10 rows available. Increase rows selector to view more rows.				

Figure 3: Sample records – HOTEL table

Room Table

SELECT * FROM ROOM;

ROOM_ID	ROOM_NUMBER	ROOM_TYPE	PRICE	STATUS	HOTEL_ID
101	A101	Single	1500	Available	1
102	A102	Double	2500	Occupied	1
103	A103	Deluxe	3500	Available	2
104	A104	Suite	5000	Available	2
105	A105	Double	2500	Occupied	3
106	A106	Single	1500	Available	3
107	A107	Deluxe	3500	Maintenance	4
108	A108	Suite	5000	Available	4
109	A109	Double	2500	Available	5
110	A110	Single	1500	Occupied	5
More than 10 rows available. Increase rows selector to view more rows.					

Figure 4: Sample records – ROOM table

Staff Table

SELECT * FROM STAFF;

STAFF_ID	STAFF_NAME	PHONE	ROLE	SALARY	HOTEL_ID
1	Ramesh	9100000001	Manager	40000	1
2	Suresh	9100000002	Receptionist	20000	1
3	Kavya	9100000003	Accountant	25000	2
4	Priyanka	9100000004	Manager	45000	2
5	Mahesh	9100000005	Receptionist	20000	3
6	Lakshmi	9100000006	Housekeeping	15000	3
7	Naveen	9100000007	Manager	40000	4
8	Swathi	9100000008	Receptionist	22000	4
9	Karthik	9100000009	Security	18000	5
10	Deepa	9100000010	Housekeeping	16000	5
More than 10 rows available. Increase rows selector to view more rows.					

Figure 5: Sample records – STAFF table

Reservation Table

SELECT * FROM RESERVATION;

RESERVATION_ID	NAME	ROOM_NUMBER	CHECK_IN_DATE	CHECK_OUT_DATE	BOOKING_STATUS
1	Ravi	A101	10-SEP-26	12-SEP-26	Confirmed
2	Sita	A102	11-SEP-26	13-SEP-26	Confirmed
3	Rahul	A103	12-SEP-26	14-SEP-26	Confirmed
4	Priya	A104	13-SEP-26	15-SEP-26	Confirmed
5	Kiran	A105	14-SEP-26	16-SEP-26	Confirmed
6	Anu	A106	15-SEP-26	17-SEP-26	Confirmed
7	Arun	A107	16-SEP-26	18-SEP-26	Cancelled
8	Divya	A108	17-SEP-26	19-SEP-26	Confirmed
9	Sai	A109	18-SEP-26	20-SEP-26	Confirmed
10	Sneha	A110	19-SEP-26	21-SEP-26	Confirmed
More than 10 rows available. Increase rows selector to view more rows.					

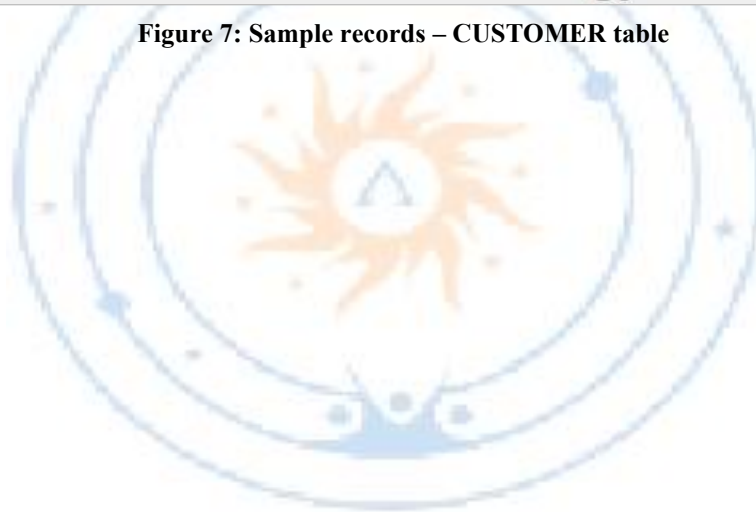
Figure 6: Sample records – RESERVATION table

Customer Table

```
SELECT * FROM CUSTOMER;
```

CUSTOMER_ID	NAME	PHONE	EMAIL	ADDRESS
1	Ravi	9000000001	ravi@gmail.com	Hyderabad
2	Sita	9000000002	sita@gmail.com	Vijayawada
3	Rahul	9000000003	rahul@gmail.com	Guntur
4	Priya	9000000004	priya@gmail.com	Chennai
5	Kiran	9000000005	kiran@gmail.com	Vizag
6	Anu	9000000006	anu@gmail.com	Tirupati
7	Arun	9000000007	arun@gmail.com	Warangal
8	Divya	9000000008	divya@gmail.com	Nellore
9	Sai	9000000009	sai@gmail.com	Kurnool
10	Sneha	9000000010	sneha@gmail.com	Mumbai

Figure 7: Sample records – CUSTOMER table



9. SQL QUERIES & SCREENSHOTS

The following queries demonstrate retrieval of customer, room, reservation, payment and staff information using selection, filtering, joins, aggregation and grouping. The outputs shown are from the sample records inserted in the database.

Q1. Display all customer details

```
SELECT * FROM CUSTOMER;
```

Output:

CUSTOMER_ID	NAME	PHONE	EMAIL	ADDRESS
1	Ravi	9000000001	ravi@gmail.com	Hyderabad
2	Sita	9000000002	sita@gmail.com	Vijayawada
3	Rahul	9000000003	rahul@gmail.com	Guntur
4	Priya	9000000004	priya@gmail.com	Chennai
5	Kiran	9000000005	kiran@gmail.com	Vizag
6	Anu	9000000006	anu@gmail.com	Tirupati
7	Arun	9000000007	arun@gmail.com	Warangal
8	Divya	9000000008	divya@gmail.com	Nellore
9	Sai	9000000009	sai@gmail.com	Kurnool
10	Sneha	9000000010	sneha@gmail.com	Mumbai

Figure 8: Output of Q1

Q2. Display all available rooms

```
SELECT Room_ID, Room_Number,  
       Room_Type, Price  
FROM ROOM  
WHERE Status = 'Available';
```

Output:

ROOM_ID	ROOM_NUMBER	ROOM_TYPE	PRICE
101	A101	Single	1500
103	A103	Deluxe	3500
104	A104	Suite	5000
106	A106	Single	1500
108	A108	Suite	5000
109	A109	Double	2500
111	A111	Deluxe	3500
113	A113	Double	2500
114	A114	Single	1500

Figure 9: Output of Q2

Q3. Display customer names and reservation status

```
SELECT c.Name,  
       r.Reservation_ID,  
       r.Booking_Status  
FROM CUSTOMER c  
JOIN RESERVATION r  
ON c.Customer_ID = r.Customer_ID;
```

Output:

NAME	RESERVATION_ID	BOOKING_STATUS
Ravi	1	Confirmed
Sita	2	Confirmed
Rahul	3	Confirmed
Priya	4	Confirmed
Kiran	5	Confirmed
Anu	6	Confirmed
Arun	7	Cancelled
Divya	8	Confirmed
Sai	9	Confirmed
Sneha	10	Confirmed

Figure 10: Output of Q3

Q4. Display reservation details with customer and room information

```
SELECT r.Reservation_ID,  
       c.Name,  
       rm.Room_Number,  
       r.Check_In_Date,  
       r.Check_Out_Date,  
       r.Booking_Status  
FROM RESERVATION r  
JOIN CUSTOMER c  
ON r.Customer_ID = c.Customer_ID  
JOIN ROOM rm  
ON r.Room_ID = rm.Room_ID;
```

Output:

RESERVATION_ID	NAME	ROOM_NUMBER	CHECK_IN_DATE	CHECK_OUT_DATE	BOOKING_STATUS
1	Ravi	A101	10-SEP-26	12-SEP-26	Confirmed
2	Sita	A102	11-SEP-26	13-SEP-26	Confirmed
3	Rahul	A103	12-SEP-26	14-SEP-26	Confirmed
4	Priya	A104	13-SEP-26	15-SEP-26	Confirmed
5	Kiran	A105	14-SEP-26	16-SEP-26	Confirmed
6	Anu	A106	15-SEP-26	17-SEP-26	Confirmed
7	Arun	A107	16-SEP-26	18-SEP-26	Cancelled
8	Divya	A108	17-SEP-26	19-SEP-26	Confirmed
9	Sai	A109	18-SEP-26	20-SEP-26	Confirmed
10	Sneha	A110	19-SEP-26	21-SEP-26	Confirmed
More than 10 rows available. Increase rows selector to view more rows.					

Figure 11: Output of Q4

Q5. Find the total paid amount

```
SELECT SUM(Amount) AS Total_Paid  
FROM PAYMENT  
WHERE Payment_Status = 'Paid';
```

Output:

TOTAL_PAID
-

Figure 12: Output of Q5

The result is empty (-) because no payment records with status 'Paid' have been inserted into the PAYMENT table yet.

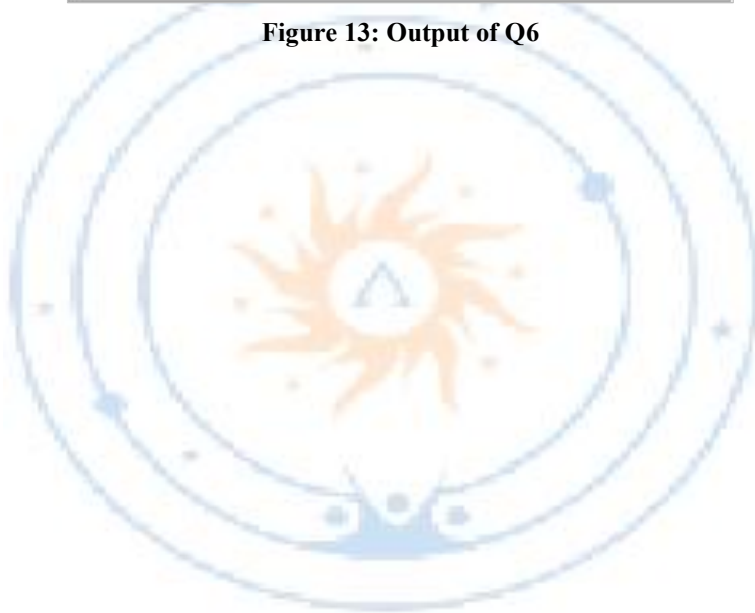
Q6. Count rooms by room type

```
SELECT Room_Type,  
       COUNT(*) AS Total_Rooms  
FROM ROOM  
GROUP BY Room_Type;
```

Output:

ROOM_TYPE	TOTAL_ROOMS
Suite	3
Single	4
Deluxe	4
Double	4

Figure 13: Output of Q6



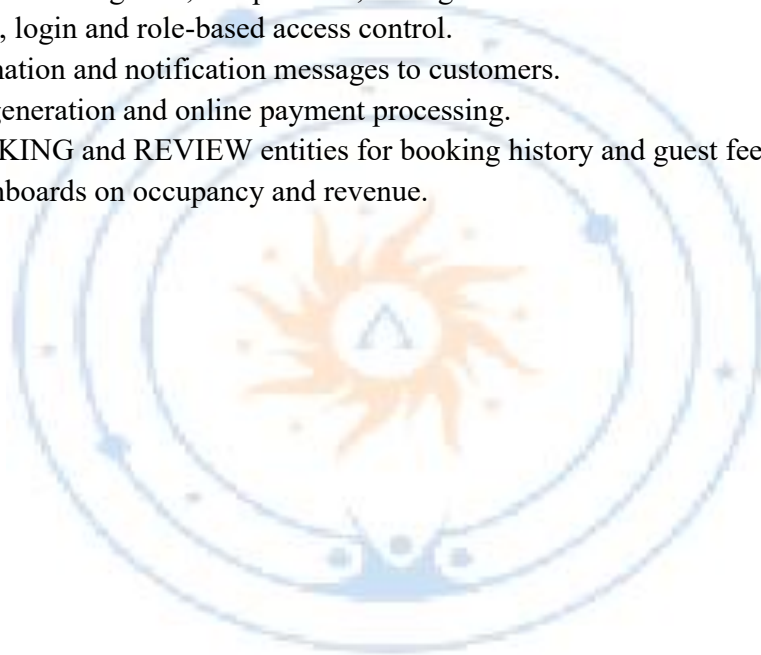
10. CONCLUSION

The Hotel Reservation Management System simplifies the process of booking and managing hotel rooms. It stores customer, room, reservation and payment details efficiently in a relational database, reduces manual work and avoids booking errors. Staff can check room availability quickly, and the database provides accurate and organized information.

The design, normalized up to 3NF and implemented in Oracle with primary keys, foreign keys and constraints, supports data integrity, and the SQL queries demonstrate retrieval, joins and aggregation over the stored data. The system improves customer service, saves time, and makes hotel management more efficient and reliable.

11. FUTURE ENHANCEMENTS

- Front-end application for guests, receptionists, managers and administrators.
- User registration, login and role-based access control.
- Booking confirmation and notification messages to customers.
- Automated bill generation and online payment processing.
- Additional BOOKING and REVIEW entities for booking history and guest feedback.
- Reports and dashboards on occupancy and revenue.



REFERENCES

1. Korth, H. F., Silberschatz, A., and Sudarshan, S., *Database System Concepts*, McGraw Hill. Available at: <https://www.mheducation.com/highered/product/Database-System-Concepts-Silberschatz.html>
2. Elmasri, R. and Navathe, S. B., *Fundamentals of Database Systems*, Pearson. Available at: <https://www.pearson.com/en-ca/subject-catalog/p/fundamentals-of-database-systems/P200000003546/9780137502523>
3. Oracle Corporation, *Oracle Database SQL Language Reference*. Available at: <https://docs.oracle.com/en/database/oracle/oracle-database/21/sqlrf/index.html>

